

Your Compiler Is Leaking Your Model: Measuring and Verifying Confidentiality Across Deep-Learning Compilation

Quinn Dougherty

*Apart Research — Secure Program
Synthesis Fellowship
quinn@for-all.dev*

Kapedalex (full name TBD)

*Apart Research — Secure Program
Synthesis Fellowship
tbd@example.com*

Siddharth Priya

*Apart Research — Secure Program
Synthesis Fellowship
tbd@example.com*

Shefali Sharma

*Apart Research — Secure Program Synthesis Fellowship
tbd@example.com*

John Wu

*Apart Research — Secure Program Synthesis Fellowship
tbd@example.com*

Abstract—Abstract to be written from the outline above.

Index Terms—compiler security, side channels, machine learning, MLIR, translation validation, non-interference

I. INTRODUCTION

- Hook: the compiler is inside every deployed model’s trust boundary, and nobody audits it for confidentiality — it can *introduce* leaks the source never had, and (surprise finding) *remove* leaks the source did have.
- The gap: side-channel work targets hand-written crypto kernels; verified compilation targets correctness, not secrecy; DL compilers (Inductor, MLIR pipelines, Polygeist) have neither.
- Two-sided thesis of the paper:
 - **Measure** — a differential eager-vs-compiled methodology attributes each observed channel to source or compiler (the 2×2 , docs/research/leak_check.methodology.siddharth.md).
 - **Verify** — per-pass checked specifications where a lowering is safe only if it both computes the same thing *and* preserves non-interference (prototypes/fc vd_ct).
- Contributions list (draft):
 - 1) A unified threat model: observer rings O0–O7 $\times$ leaked-asset axis, composing four independently developed frameworks (docs/research/threat-models.agents.md).
 - 2) The differential measurement rig and activation-function corpus, with the exp channel **compiler-removed** as headline (prototypes/leak_check).
 - 3) Two-property translation validation (“safe = congruent + non-interfering”) instantiated on Polygeist’s eight-step pipeline, with SMT semantics for the memref→LLVM slice (prototypes/fc vd_ct).
 - 4) Negative-space honesty: the count-channel confound, retired probes, and model-blocked steps reported as first-class results.
- Roadmap paragraph.

II. THREAT MODEL

- Four frameworks compose; we do not pick one (leak class $\times$ observer ring; evidence layers A–D; evidence levels L0–L4; the differential 2×2 attribution).
- The observer-ring vocabulary (canonical grid: prototypes/fc vd_ct/artifact/attacker_profile.py):
 - O0 public transcript $\rightarrow$ O7 invasive observer, ordered by directness of reach; “proved at O2, carried outward to O1” is meaningful because of the ordering; O6-and-inward is moot rather than uncovered.
- The asset axis the rings do not carry: weights (static — repetition wins) vs activations/prompts (fresh per query — one-shot channels only) vs structure (sparsity, shapes, architecture — leaked by control structure, not values).
- What “secure” means in this paper: a claim is fully specified only with all four coordinates (observer, evidence kind, evidence level, attribution).
- Scope / non-goals: integrity of weights, TEE partitioning, and running the compiled model are out (issues 29, 34, 36).

III. MEASURING: COMPILER-INTRODUCED AND COMPILER-REMOVED CHANNELS

A. Methodology

- Differential eager-vs-Inductor on deterministic channels: instruction counts (Ir), branch counts (Bc), instruction-level taint; baited input classes probe value-regime branches.
- The 2×2 attribution: whose fault — source or compiler; four-valued verdicts (OBLIVIOUS / DISTINGUISHABLE / COMPILER-INTRODUCED / COMPILER-REMOVED).
- Instrument honesty: the Zen5 count-channel confound; verdicts resting on small $|dIr|$ without taint corroboration.

ration were retired (softmax), criterion updated in noninterference.py.

B. Results

- Activation corpus (CSI-NN mechanism 2):
 - relu: OBLIVIOUS both builds — PyTorch’s eager kernel is already a vectorized max; library kernels can be constant-time where hand code is not.
 - **exp: COMPILER-REMOVED (headline)**. Eager libm expf branches on magnitude (dfr $\approx$ +35M, taint fires); Inductor lowers to a branchless vectorized polynomial — compilation erased the CSI-NN channel.
 - softmax: RETIRED — degenerate probe (uniform rows cancel under max-subtraction); shown as a worked example of the retirement discipline.
- Freezing / constant-folding: instruction-level taint proof of the weight-fold leak (probe branch work, PR 55 lineage).
- Autotune: max-autotune GEMM kernel **selection** as a value-dependent channel.
- Overnight-sweep coverage numbers.

IV. VERIFYING: TWO-PROPERTY TRANSLATION VALIDATION FOR A REAL PIPELINE

- Why Polygeist: most translatable corpus of the six compilers mapped (61.9% of operation mentions vs 53.4% HEIR, 17.3% torch-mlir); speaks the shared dialects (affine, scf, memref, arith) — the compiler most likely to be verified end-to-end rather than in patches.
- The property: a lowering is VERIFIED only if **both halves** hold — semantic congruence (computes the same thing) and non-interference (self-composition; hole congruence includes poison bits).
- The method: one checked specification per pass, transcribed from the pass source with a citation, each preserving verdict paired with a falsifying control; append-only journal; coverage counter re-checks every verdict.
- Pipeline status table (regenerate from live data at submission): eight steps from tools/cgeist/driver.cc — mem2reg, loop-restructure, affine-cfg, canonicalize-for, lower-affine, parallel-lower, scf-to-openmp, polygeist-to-llvm; note which are checked vs model-blocked (the two parallel steps).
- Scaling the check: the taint prefilter before the solver; SMT semantics for LLVM memory ops closing the memref→LLVM slice.
- Per-program vs per-pass proofs: macro-templates cover a translation only if both halves hold; affine.load/store with identity maps as the cheapest-and-most-used translations.

V. CASE STUDIES AND EVALUATION

- KyberSlash through the non-interference lens: what the proofs_l2_seabmc jobs admit and reject (prototypes/

proofs_l2_seabmc, commit 8e0cf35 “five more non-interference jobs, and what KyberSlash admits”).

- End-to-end walk of one program through the verified Polygeist steps, reporting the four coordinates (observer, evidence kind, level, attribution) at each stage.
- Structure leaks as a distinct class: sparsity and dynamic shapes (prototypes/mlir_leak, prototypes/Staging_NI) — leaked by control structure, not values.
- Coverage accounting: steps checked / model-blocked, unproved operation counts, sweep breadth — the numbers the coverage counter regenerates.

VI. RELATED WORK

- Side-channel extraction of model internals: CSI-NN [1] and successors; where our asset/observer grid places them.
- Timing leaks in compiled crypto: KyberSlash [2]; constant-time verification (ct-verif, Binsec/Rel lineage) — hand kernels, not compilers.
- Compiler correctness: CompCert, Alive2, translation validation [3] — correctness without secrecy; our two-property extension.
- Secure compilation / full abstraction — the theory frame our per-pass specs approximate.
- MLIR [4] and Polygeist [5] as substrate; MLIR-TV and dialect-level verification efforts (docs/research/mlir-tv-custom-dialect.agents.md).
- IFC dialects claim-check (“first MLIR IFC dialect”, issue 23).

VII. DISCUSSION AND LIMITATIONS

- What the evidence does not reach: measured results are machine- and version-specific (Zen5 confound as the worked example); proofs are in a model, not on silicon (layers A–D vocabulary keeps these apart).
- Model-blocked steps: the two parallel Polygeist passes; what unblocking would take.
- The compiler-removed surprise: compilation is not monotonically bad for confidentiality — implications for “compile with hardening” guidance.
- Generalizing past Polygeist/Inductor: what transfers via the shared dialects, what is compiler-specific.
- Ethics: no deployed-system exploitation; all measurement on our own hardware and models.

VIII. CONCLUSION

- Restate the two-sided result: the same threat model prices both the attack measurements and the verification claims.
- The discipline is the contribution as much as the results: four-coordinate claims, falsifying controls, retired probes.
- Future work: remaining Polygeist steps, per-program proofs at scale, other compilers on the map (HEIR, torch-mlir), egress-side defenses (issues 26–30).

REFERENCES

- [1] L. Batina, S. Bhasin, D. Jap, and S. Picek, “CSI NN: Reverse Engineering of Neural Network Architectures Through Electromagnetic Side Channel,” in *28th USENIX Security Symposium*, 2019.
- [2] D. J. Bernstein *et al.*, “KyberSlash: Exploiting secret-dependent division timings in Kyber implementations.” 2024.
- [3] N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, and J. Regehr, “Alive2: Bounded Translation Validation for LLVM,” in *42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, 2021.
- [4] C. Lattner *et al.*, “MLIR: Scaling Compiler Infrastructure for Domain Specific Computation,” in *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021.
- [5] W. S. Moses, L. Chelini, R. Zhao, and O. Zinenko, “Polygeist: Raising C to Polyhedral MLIR,” in *30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2021.